

Task 1: EVD and SVD of square images

Step 1 — Convert image to grayscale matrix $A \in \mathbb{R}^{n \times n}$

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

def show(img, title):
    plt.imshow(np.clip(img, 0, 255), cmap="gray")
    plt.title(title); plt.axis("off"); plt.show()

def load_gray(path):
    img = Image.open(path).convert("RGB")
    pixels = np.array(img, dtype=float)
    r = pixels[:, :, 0]
    g = pixels[:, :, 1]
    b = pixels[:, :, 2]
    gray = 0.299*r + 0.587*g + 0.114*b    # standard luminosity weights
    return gray

A = load_gray("Cat_Square.png")
print("A shape:", A.shape)
show(A, "grayscale square image")

A shape: (512, 512)
```

grayscale square image



Step 2 — Perform EVD: $A = Q \Lambda Q^{-1}$

```
eigvals, Q = np.linalg.eig(A)    # eigvals = diagonal of Lambda, Q =  
eigenvectors as columns  
Q_inv = np.linalg.inv(Q)
```

Step 3 — Perform SVD: $A = U \Sigma V^t$

```
# V = A_transpose * A  
vv, V = np.linalg.eig(A.T @ A)  
vv, V = np.real(vv), np.real(V)  
order_v = np.argsort(-vv)  
vv, V = vv[order_v], V[:, order_v]  
  
# U = A * A_transpose  
vu, U = np.linalg.eig(A @ A.T)  
vu, U = np.real(vu), np.real(U)  
order_u = np.argsort(-vu)  
vu, U = vu[order_u], U[:, order_u]  
  
sigma = np.sqrt(np.clip(vv, 0, None))    # singular values = sqrt of  
eigenvalues  
  
# fix sign so A*v = sigma*u actually holds (eig() gives arbitrary  
sign)
```

```

for i in range(len(sigma)):
    if sigma[i] > 1e-10:
        expected_u = (A @ V[:, i]) / sigma[i]
        if np.dot(expected_u, U[:, i]) < 0:
            U[:, i] = -U[:, i]

```

Step 4 — Choose k, handling EVD's complex conjugate pairs

```

def find_partner(vals, i, taken):
    target = np.conj(vals[i])
    best_j, best_diff = -1, 1e9
    for j in range(len(vals)):
        if j in taken:
            continue
        diff = abs(vals[j] - target)
        if diff < best_diff:
            best_diff, best_j = diff, j
    return best_j

def pick_top_k(vals, k):
    order = np.argsort(-np.abs(vals))    # sort by magnitude, biggest
    first
    chosen, taken = [], set()
    for i in order:
        if len(chosen) >= k:
            break
        if i in taken:
            continue
        chosen.append(i); taken.add(i)
        if abs(vals[i].imag) > 1e-8:    # complex eigenvalue
            j = find_partner(vals, i, taken)
            if j != -1:
                chosen.append(j); taken.add(j)
    return chosen

k_values = [10, 50, 150]

```

Step 5 — Reconstruct using EVD: $A_k = Q \Lambda_k Q^{-1}$

```

def evd_reconstruct(vals, Q, Q_inv, k):
    idx = pick_top_k(vals, k)
    lam_k = np.zeros(len(vals), dtype=complex)
    for i in idx:
        lam_k[i] = vals[i]    # keep chosen eigenvalues, rest
    stay zero
    Lambda_k = np.diag(lam_k)
    Ak = Q @ Lambda_k @ Q_inv
    return np.real(Ak)    # imaginary part is ~0 rounding
noise

```

Step 5b — Reconstruct using SVD: $A_k = U \Sigma_k V^t$

```
def svd_reconstruct(U, sigma, V, k):  
    Uk = U[:, :k]  
    Sk = np.diag(sigma[:k])  
    Vk = V[:, :k]  
    return Uk @ Sk @ Vk.T
```

Step 6 — Display original, reconstruction, error image $|A - A_k|$

```
def show(img, title):  
    plt.imshow(np.clip(img, 0, 255), cmap="gray")  
    plt.title(title); plt.axis("off"); plt.show()  
  
for k in k_values:  
    Ak_evd = evd_reconstruct(eigvals, Q, Q_inv, k)  
    Ak_svd = svd_reconstruct(U, sigma, V, k)  
  
    show(A, "original")  
    show(Ak_evd, f"EVD reconstruction k={k}")  
    show(np.abs(A - Ak_evd), f"EVD error image k={k}")  
  
    show(A, "original")  
    show(Ak_svd, f"SVD reconstruction k={k}")  
    show(np.abs(A - Ak_svd), f"SVD error image k={k}")
```

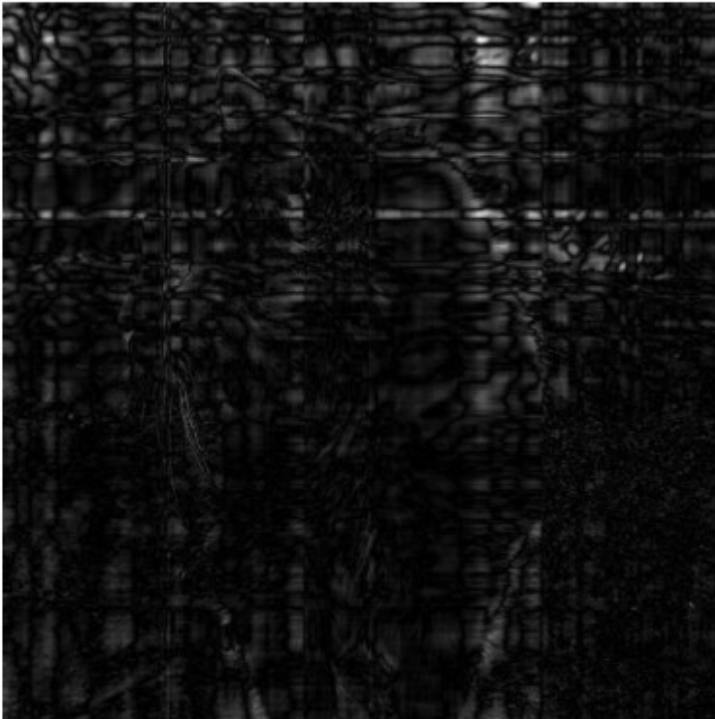
original



EVD reconstruction $k=10$



EVD error image $k=10$



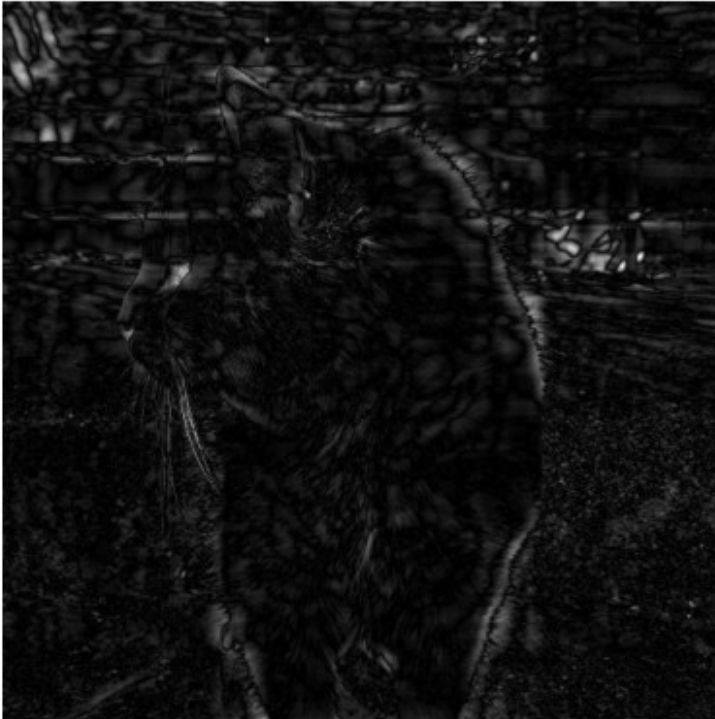
original



SVD reconstruction $k=10$



SVD error image $k=10$



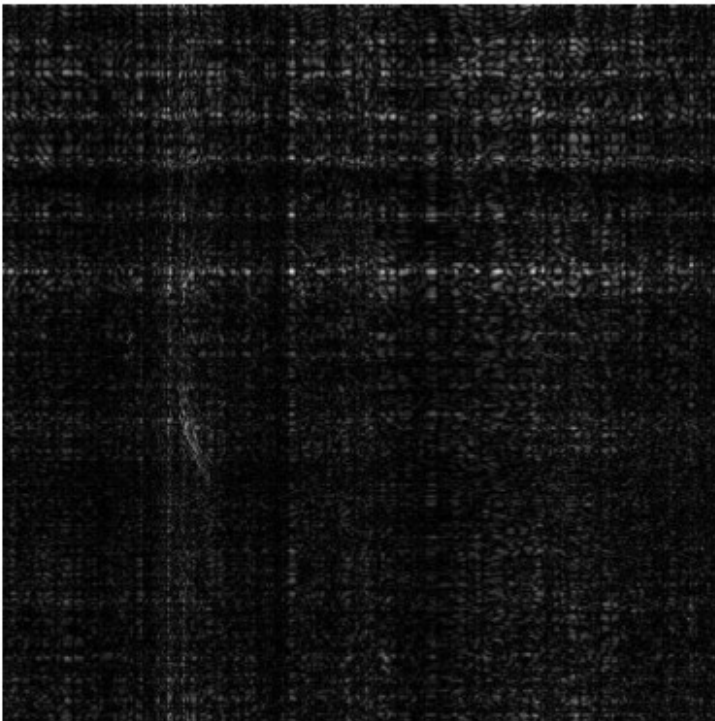
original



EVD reconstruction $k=50$



EVD error image $k=50$



original



SVD reconstruction $k=50$



SVD error image $k=50$



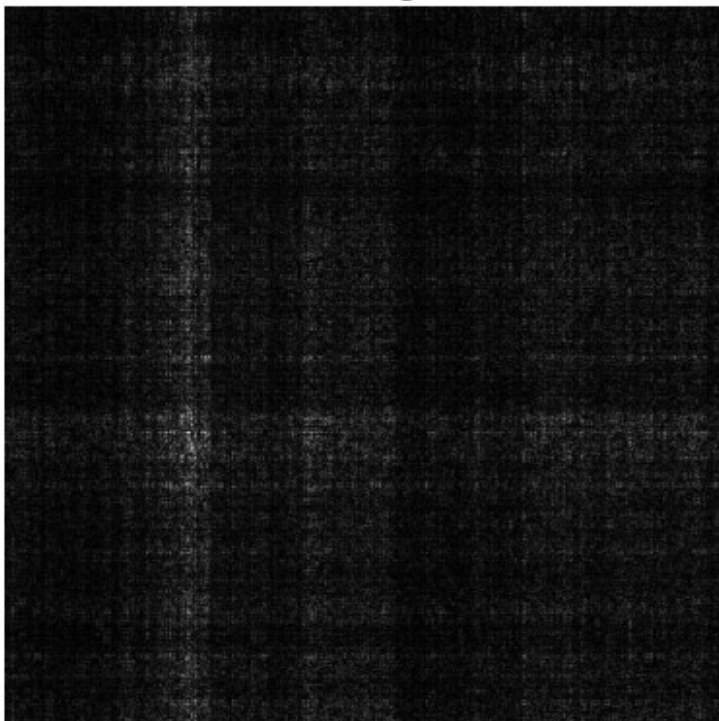
original



EVD reconstruction $k=150$



EVD error image $k=150$



original



SVD reconstruction $k=150$



SVD error image k=150



Step 7 — Frobenius norm error: $E(k) = \|A - A_k\|_F$

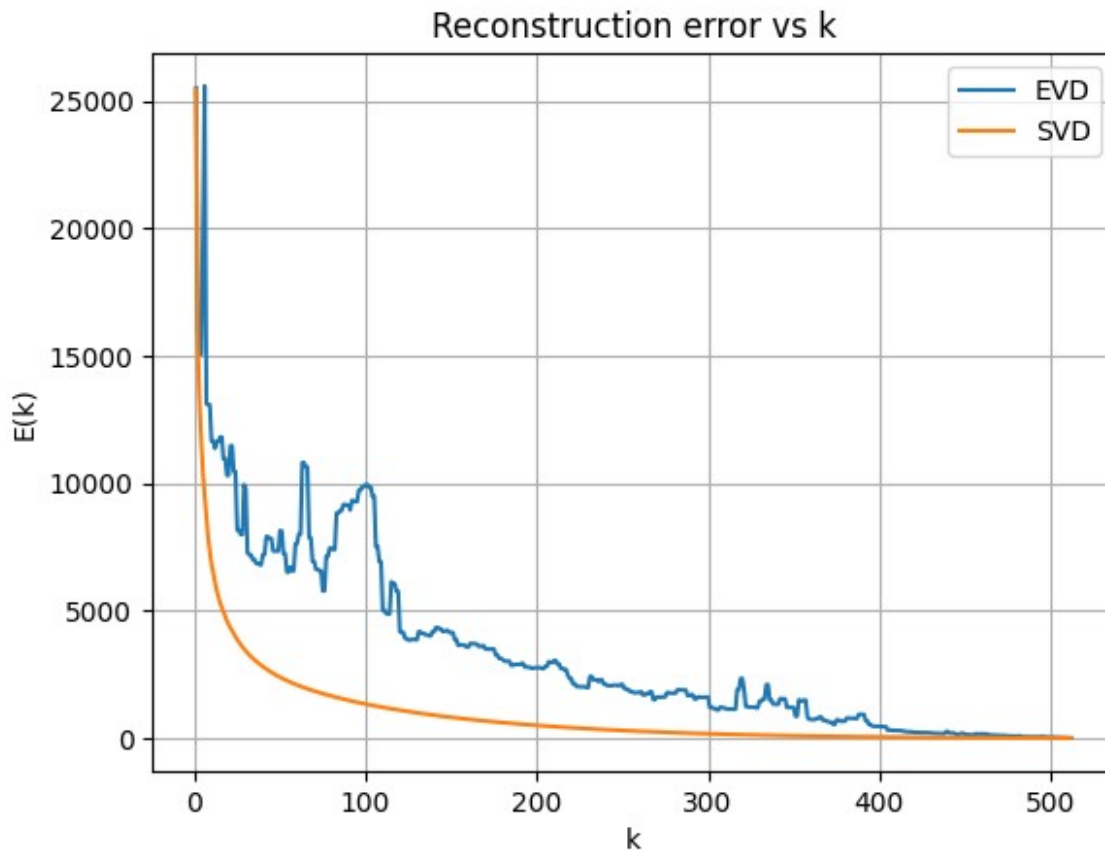
```
def frob_error(A, Ak):  
    diff = A - Ak  
    return np.sqrt(np.sum(diff * diff))  
  
for k in k_values:  
    e_evd = frob_error(A, evd_reconstruct(eigvals, Q, Q_inv, k))  
    e_svd = frob_error(A, svd_reconstruct(U, sigma, V, k))  
    print("k =", k, " EVD E(k) =", round(e_evd, 2), " SVD E(k) =",  
round(e_svd, 2))
```

```
k = 10  EVD E(k) = 11649.56  SVD E(k) = 6824.41  
k = 50  EVD E(k) = 8128.1   SVD E(k) = 2385.82  
k = 150 EVD E(k) = 4138.29  SVD E(k) = 809.94
```

Step 8 — Plot $E(k)$ vs k

```
n = A.shape[0]  
ks = list(range(1, n + 1))  
evd_err, svd_err = [], []  
  
for k in ks:  
    evd_err.append(frob_error(A, evd_reconstruct(eigvals, Q, Q_inv,  
k)))  
    svd_err.append(frob_error(A, svd_reconstruct(U, sigma, V, k)))
```

```
plt.plot(ks, evd_err, label="EVD")
plt.plot(ks, svd_err, label="SVD")
plt.xlabel("k"); plt.ylabel("E(k)")
plt.title("Reconstruction error vs k")
plt.legend(); plt.grid(True); plt.show()
```



Step 9 — Compare EVD vs SVD, same k

```
print("k      EVD_E(k)      SVD_E(k)")
for k in k_values:
    e1 = frob_error(A, evd_reconstruct(eigvals, Q, Q_inv, k))
    e2 = frob_error(A, svd_reconstruct(U, sigma, V, k))
    print(k, round(e1,2), round(e2,2))
```

Task 2: SVD on rectangular image

Step 1 — Grayscale conversion

```
B = load_gray("Cat_Rectangle.png")
print("B shape:", B.shape)
show(B, "grayscale rectangular image")
```

B shape: (600, 450)

grayscale rectangular image



Step 2 — SVD: $A = U \Sigma V^t$

```
vv2, V2 = np.linalg.eig(B.T @ B)
vv2, V2 = np.real(vv2), np.real(V2)
order1 = np.argsort(-vv2)
vv2, V2 = vv2[order1], V2[:, order1]

vu2, U2 = np.linalg.eig(B @ B.T)
vu2, U2 = np.real(vu2), np.real(U2)
order2 = np.argsort(-vu2)
vu2, U2 = vu2[order2], U2[:, order2]

sigma2 = np.sqrt(np.clip(vv2, 0, None))

for i in range(len(sigma2)):
    if sigma2[i] > 1e-10:
        expected = (B @ V2[:, i]) / sigma2[i]
        if np.dot(expected, U2[:, i]) < 0:
            U2[:, i] = -U2[:, i]
```

Step 3 — Rank-k approximation: $A_k = U \Sigma_k V^t$

```
for k in [10, 50, 150]:  
    Bk = svd_reconstruct(U2, sigma2, V2, k)  
    e = frob_error(B, Bk)  
    show(B, "original")  
    show(Bk, f"SVD reconstruction k={k}")  
    show(np.abs(B - Bk), f"error image k={k}")  
    print("k =", k, " E(k) =", round(e, 2))
```

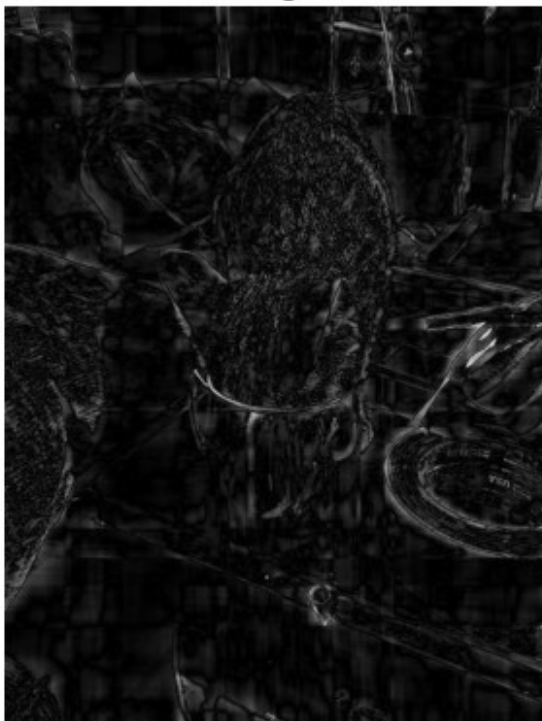
original



SVD reconstruction $k=10$



error image $k=10$



$k = 10$ $E(k) = 10397.45$

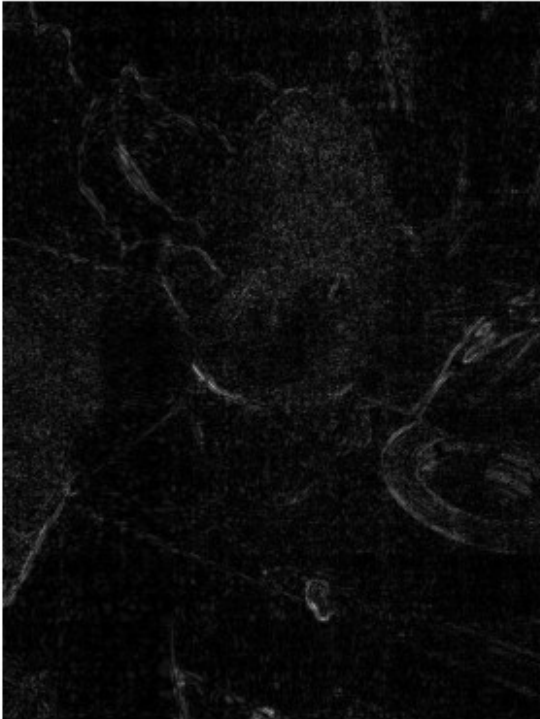
original



SVD reconstruction $k=50$



error image k=50



k = 50 E(k) = 4777.32

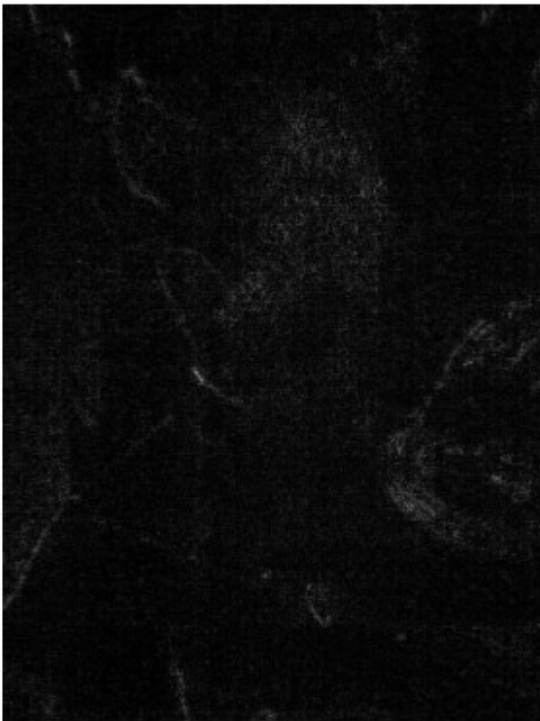
original



SVD reconstruction $k=150$



error image $k=150$



$k = 150$ $E(k) = 1688.02$

Step 4 — Error vs k

```
m = min(B.shape)
ks2 = list(range(1, m + 1))
rect_err = [frob_error(B, svd_reconstruct(U2, sigma2, V2, k)) for k in ks2]

plt.plot(ks2, rect_err)
plt.xlabel("k"); plt.ylabel("E(k)")
plt.title("SVD error vs k - rectangular image")
plt.grid(True); plt.show()
```

